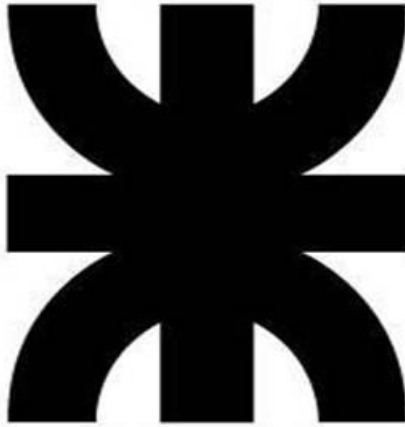




Universidad Tecnológica Nacional
Facultad Regional Concepción del Uruguay

Ingeniería en Sistemas de Información

Base de datos

Informe TP integrador

Alumno:

- ***Benay Franco***
- ***Danielis Mateo***
- ***Oliva Josue Emanuel***
- ***Poerio Santiago***
- ***Prediger Misael***

Docentes:

- ***Ing. Pablo Colombo***
- ***Ing. Francisco Fazzio***

Introducción: descripción del problema y objetivos.

Descripción del Problema

Las bibliotecas manejan grandes volúmenes de datos transaccionales (socios, libros, préstamos y sanciones). Sin embargo, la explotación de esta información tradicionalmente requiere conocimientos técnicos avanzados en SQL o depende de reportes rígidos y predefinidos. Esto genera dos problemas principales: la inaccesibilidad de los datos para usuarios no técnicos (como el personal administrativo) y la falta de personalización para el socio, al no aprovechar el historial de lectura para ofrecer recomendaciones automatizadas.

Para resolver esto, se propone BibliolA : una solución que combina una base de datos relacional robusta en MySQL con Inteligencia Artificial para permitir la interacción mediante lenguaje natural y ofrecer sugerencias de lectura personalizadas.

Objetivos del Proyecto

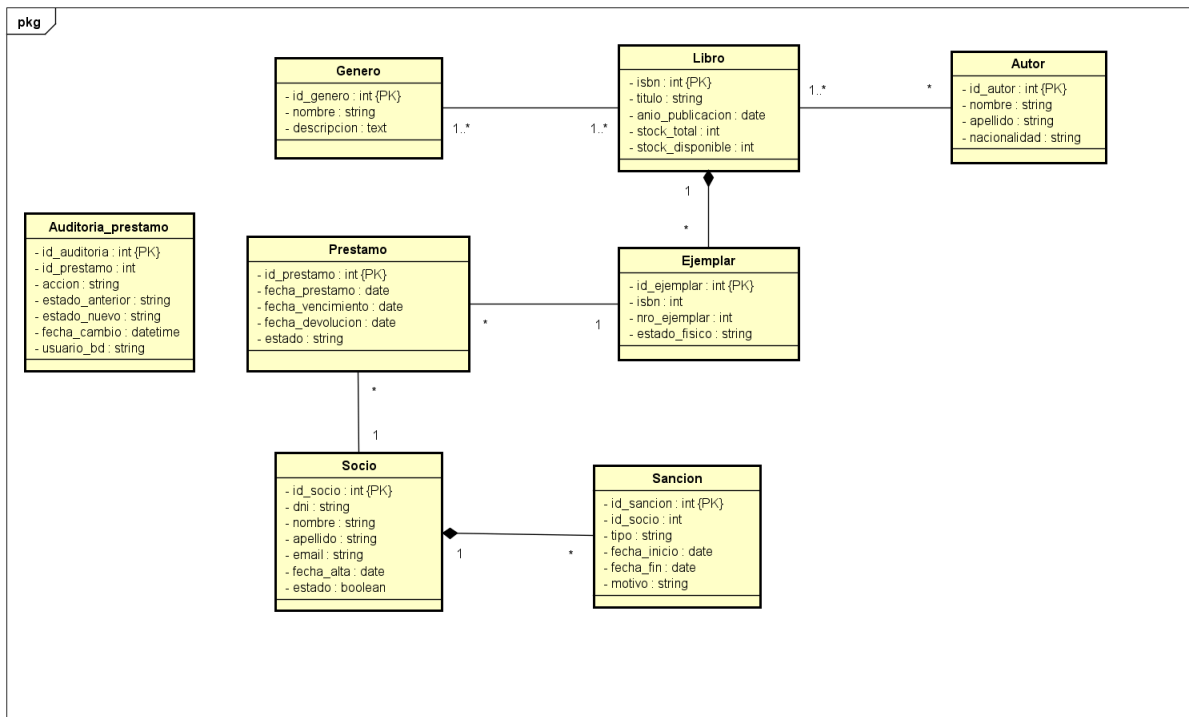
Objetivo General

Desarrollar una aplicación funcional que integre una base de datos relacional con un agente de Inteligencia Artificial capaz de traducir preguntas en lenguaje natural a consultas SQL y generar recomendaciones personalizadas sobre el dominio de una biblioteca.

Objetivos Específicos

- Diseño del Esquema: Implementar la base de datos aplicando reglas de integridad, claves foráneas, restricciones e índices de búsqueda eficiente.
- Automatización del Negocio: Crear procedimientos almacenados y *triggers* para el control automatizado de stock , penalizaciones y auditoría de cambios.
- Componente Text-to-SQL: Desarrollar el agente inteligente para interpretar preguntas en español y ejecutarlas como consultas SQL válidas.
- Módulo de Recomendaciones: Implementar un algoritmo que analice las preferencias e historial del socio para sugerir libros disponibles con justificaciones generadas por la IA.

Modelo Entidad-Relación (DER) completo con notación estándar.



Modelo Relacional (esquema de tablas con tipos y constraints).

GENERO (

- id_genero: INT AUTO_INCREMENT {PK}
- nombre: VARCHAR(100) NOT NULL UNIQUE
- descripcion: TEXT NULL

AUTOR (

- id_autor: INT AUTO_INCREMENT {PK}
- nombre: VARCHAR(100) NOT NULL
- apellido: VARCHAR(100) NOT NULL
- nacionalidad: VARCHAR(100) NOT NULL

LIBRO (

- isbn: VARCHAR(20) {PK}
- titulo: VARCHAR(255) NOT NULL
- anio_publicacion: INT NULL
- stock_total: INT NOT NULL
- stock_disponible: INT NOT NULL

- *Restricciones:* chk_stock (stock_disponible >= stock_total),
chk_stock_positivo (stock_disponible >= 0)
)

SOCIO (

- id_socio: INT AUTO_INCREMENT {PK}
- dni: VARCHAR(15) **NOT NULL UNIQUE**
- nombre: VARCHAR(100) **NOT NULL**
- apellido: VARCHAR(100) **NOT NULL**
- email: VARCHAR(150) **NOT NULL UNIQUE**
- fecha_alta: DATE **NOT NULL**
- estado: VARCHAR(20) **NOT NULL DEFAULT 'ACTIVO'**
- *Restricciones:* chk_estado_socio (estado IN ('ACTIVO', 'SUSPENDIDO', 'BAJA'))
)

LIBRO_AUTOR (

- isbn: VARCHAR(20) {PK} {FK} → Referencia a LIBRO(isbn) **ON DELETE CASCADE ON UPDATE CASCADE**
- id_autor: INT {PK} {FK} → Referencia a AUTOR(id_autor) **ON DELETE CASCADE ON UPDATE CASCADE**
)

LIBRO_GENERO (

- isbn: VARCHAR(20) {PK} {FK} → Referencia a LIBRO(isbn) **ON DELETE CASCADE ON UPDATE CASCADE**
- id_genero: INT {PK} {FK} → Referencia a GENERO(id_genero) **ON DELETE CASCADE ON UPDATE CASCADE**
)

EJEMPLAR (

- id_ejemplar: INT AUTO_INCREMENT {PK}
- isbn: VARCHAR(20) {FK} → Referencia a LIBRO(isbn) **ON DELETE RESTRICT ON UPDATE CASCADE**
- nro_ejemplar: INT **NOT NULL**
- estado_fisico: VARCHAR(20) **NOT NULL DEFAULT 'DISPONIBLE'**
- *Restricciones:* chk_estado_ejemplar (estado_fisico IN ('DISPONIBLE', 'PRESTADO', 'BAJA'))
)

SANCION (

- id_sancion: INT AUTO_INCREMENT **{PK}**
 - id_socio: INT **{FK}** → Referencia a SOCIO(id_socio) **ON DELETE CASCADE ON UPDATE CASCADE**
 - tipo: VARCHAR(50) **NOT NULL**
 - fecha_inicio: DATE **NOT NULL**
 - fecha_fin: DATE **NOT NULL**
 - motivo: TEXT **NOT NULL**
-)

PRESTAMO (

- id_prestamo: INT AUTO_INCREMENT **{PK}**
 - id_socio: INT **{FK}** → Referencia a SOCIO(id_socio) **ON DELETE RESTRICT ON UPDATE CASCADE**
 - id_ejemplar: INT **{FK}** → Referencia a EJEMPLAR(id_ejemplar) **ON DELETE RESTRICT ON UPDATE CASCADE**
 - fecha_prestamo: DATE **NOT NULL**
 - fecha_vencimiento: DATE **NOT NULL**
 - fecha_devolucion: DATE **NULL**
 - estado: VARCHAR(20) **NOT NULL DEFAULT 'ACTIVO'**
 - *Restricciones:* chk_estado_prestamo (estado IN ('ACTIVO', 'DEVUELTO', 'VENCIDO'))
-)

AUDITORIA_PRESTAMOS (

- id_auditoria: INT AUTO_INCREMENT **{PK}**
 - id_prestamo: INT **NOT NULL**
 - accion: VARCHAR(10) **NOT NULL** (Valores: 'INSERT', 'UPDATE', 'DELETE')
 - estado_anterior: VARCHAR(20) **NULL**
 - estado_nuevo: VARCHAR(20) **NULL**
 - fecha_cambio: DATETIME **NOT NULL**
 - usuario_bd: VARCHAR(100) **NOT NULL**
-)

Decisiones de diseño: justificación de tipos de datos, índices, políticas de FK.

Decisiones de Diseño

Justificación de Tipos de Datos

La selección de los tipos de datos en el esquema de BibliolA se realizó bajo criterios de optimización de espacio en disco, rendimiento de consultas y coherencia con el dominio de negocio:

- **Identificadores (IDs) y Claves (INT):** Para todas las claves primarias numéricas y autonómicas (como `id_socio`, `id_prestamo`, `id_ejemplar`, `id_sancion`) se utilizó el tipo `INT`. Esto garantiza registros únicos ocupando únicamente 4 bytes por fila, agilizando además las operaciones de indexación y combinación de tablas (`JOIN`).
- **Manejo de Cadenas de Texto (VARCHAR vs TEXT):** Se utilizó `VARCHAR` con longitudes acotadas para atributos de tamaño predecible, tales como `VARCHAR(255)` para títulos de libros y motivos de sanción, `VARCHAR(20)` para el DNI y `VARCHAR(150)` para los correos electrónicos. Esto evita el desperdicio de memoria en el motor de almacenamiento de MySQL. El tipo `TEXT` se reservó exclusivamente para la columna `descripcion` en la tabla `Genero`, permitiendo almacenar reseñas extensas.
- **Control del Tiempo (DATE y DATETIME):** Las fechas asociadas a eventos de días calendario específicos (como `fecha_prestamo`, `fecha_vencimiento`) se definieron como `DATE`, optimizando el almacenamiento a 3 bytes. Por el contrario, para la tabla `Auditoria_prestamo`, el campo `fecha_cambio` se configuró como `DATETIME` (8 bytes) debido a la necesidad crítica de registrar con precisión de horas, minutos y segundos el momento exacto de una modificación.

Justificación de Índices

Para responder de manera eficiente a las exigencias de rendimiento del agente de Inteligencia Artificial y agilizar las búsquedas recurrentes en el sistema, se crearon los siguientes índices estratégicos:

- Índice Único en Socio(dni) y Socio(email): Al definirse como UNIQUE, se genera automáticamente un índice de tipo B-Tree. Esto acelera drásticamente las búsquedas al momento de validar la identidad de un socio durante el flujo de un préstamo, reduciendo la complejidad del costo de búsqueda en tablas grandes.
- Índice en Ejemplar(isbn) : Dado que el agente de IA requiere consultar frecuentemente el inventario cruzando ejemplares físicos con títulos de libros, la existencia de índices sobre el código isbn optimiza el rendimiento operativo de los comandos inner join y la velocidad del trigger de actualización de stock.

Políticas de Claves Foráneas (FOREIGN KEY)

Para mantener la integridad referencial y evitar la existencia de registros huérfanos, se justifican las siguientes políticas de acción ante modificaciones o eliminaciones en cascada:

- Política ON DELETE RESTRICT / ON UPDATE CASCADE en Préstamos: Las relaciones desde Socio y Ejemplar hacia la tabla Prestamo están protegidas con restricciones estrictas de borrado (RESTRICT). Esto impide que se elimine un socio o un ejemplar físico de la base de datos si cuenta con un historial de préstamos activo, protegiendo la integridad del negocio de la biblioteca. La actualización en cascada (CASCADE) asegura que si un ID se modifica, el cambio se replique automáticamente en el historial.
- Política de Composición en Libro y Ejemplar: En consistencia con el diagrama UML, si un registro maestro en la tabla Libro es dado de baja de forma definitiva del catálogo, se configuró una

política ON DELETE CASCADE para eliminar de manera automática todos sus ejemplares físicos asociados en la tabla Ejemplar.

- Desacoplamiento Completo en Auditoria_prestamo: La tabla de auditoría carece de restricciones físicas de tipo FOREIGN KEY con la tabla Prestamo. Esta decisión de diseño garantiza que un préstamo pueda ser modificado o eliminado físicamente del sistema operativo sin que la base de datos bloquee la acción, permitiendo que el disparador (TRIGGER) inserte la traza histórica de forma transparente e independiente.

Descripción de cada stored procedure y trigger: propósito, parámetros, casos que maneja.

Descripción de Stored Procedures y Triggers

Stored Procedures

sp_registrar_prestamo

- Propósito: Automatizar y asegurar el alta de un nuevo préstamo en la biblioteca, garantizando que tanto el socio como el ejemplar físico cumplan de forma estricta con los requisitos de negocio antes de consolidar la transacción.
- Parámetros:
 - p_id_socio (INT - Entrada): Identificador único del socio que solicita el libro.
 - p_id_ejemplar (INT - Entrada): Identificador de la copia física del libro que se entregará.
- Casos que maneja:
 - Validación de Socio Activo: Verifica que el estado del socio en la tabla SOCIO sea estrictamente 'ACTIVO'. Si el usuario está suspendido, interrumpe el flujo emitiendo un error .

- Límite Máximo de Préstamos: Controla que el socio no posea 3 o más préstamos activos en simultáneo. Ante este escenario, deniega la operación.
- Disponibilidad del Ejemplar: Consulta la tabla EJEMPLAR para asegurar que el estado_fisico sea igual a 'DISPONIBLE'. Si está prestado o de baja, cancela la solicitud.
- Atomicidad de la Operación: Envuelve la inserción en PRESTAMO (calculando automáticamente 14 días de vigencia) y la actualización del estado del ejemplar a 'PRESTADO' dentro de un bloque START TRANSACTION y COMMIT. Si ocurre algún fallo de base de datos, un manejador de excepciones ejecuta de forma inmediata un ROLLBACK para evitar inconsistencias de datos.

sp_registrar_devolucion

- Propósito: Registrar formalmente el retorno de un ejemplar físico a la biblioteca, restableciendo el inventario y derivando al sistema de penalizaciones en caso de incumplimiento de plazos.
- Parámetros:
 - p_id_prestamo (INT - Entrada): Identificador del préstamo activo que se desea finalizar.
- Casos que maneja:
 - Préstamo ya procesado: Comprueba si el estado del préstamo actual es 'DEVUELTO' para evitar duplicaciones o alteraciones erróneas de stock.
 - Cierre Transaccional: Ejecuta un UPDATE sobre la tabla PRESTAMO estampando la fecha corriente (CURDATE()) y modificando el estado del ejemplar a 'DISPONIBLE'.
 - Control de Mora y Penalización Automática: Calcula la diferencia en días mediante DATEDIFF() entre el día de la entrega y la fecha_vencimiento. Si el resultado es mayor a cero (mora detectada), invoca de manera automática al procedimiento subordinado sp_generar_sancion pasándole los días de retraso calculados.

sp_generar_sancion

- Propósito: Registrar una penalización formal a un socio debido a una entrega fuera de término, afectando su estado operativo en la biblioteca de manera inmediata.
- Parámetros:
 - p_id_socio (INT - Entrada): Código del socio infractor.
 - p_tipo (VARCHAR - Entrada): Descripción de la penalización.
 - p_dias_mora (INT - Entrada): Cantidad de días de retraso que sirven para ponderar la sanción.
- Casos que maneja:
 - Cálculo e Inserción: Inserta un registro en la tabla SANCION, determinando las fechas de inicio y fin de la penalización de manera proporcional a la gravedad de la mora.

Triggers

trg_audit_prestamo

- Propósito: Mantener un rastro analítico e histórico inalterable de cualquier modificación dentro de la tabla PRESTAMO.
- Casos que maneja:
 - AFTER INSERT: Tras registrarse un nuevo préstamo, inserta en Auditoria_prestamo la acción 'INSERT', dejando el estado anterior como vacío (NULL) y capturando el NEW.estado.
 - AFTER UPDATE: Si cambian el estado del préstamo (por ejemplo, de 'ACTIVO' a 'DEVUELTO'), el trigger captura simultáneamente el estado previo (OLD.estado) y el estado modificado (NEW.estado).
 - AFTER DELETE: En el caso fortuito de un borrado físico, resguarda toda la información final del registro destruido (OLD.id_prestamo y OLD.estado), guardando la acción 'DELETE'. Todos los casos registran con precisión la fecha del servidor (NOW()) y el identificador de sesión del sistema operativo (USER()).

trg_estado_socio

- Propósito: Inhabilitar al socio de forma inmediata tras recibir una sanción, bloqueando de manera nativa su capacidad operativa.
- Casos que maneja:
 - AFTER INSERT ON SANCION: Este disparador reacciona al instante en que una fila es añadida en la tabla SANCION. Ejecuta un comando UPDATE sobre la tabla SOCIO, mutando de manera transparente su columna estado al valor de suspendido . Esto garantiza que ninguna aplicación externa pueda volver a prestarle libros mientras la sanción persista.

trg_actualizar_stock_insert

- Propósito: Sincronizar de forma exacta y en tiempo real el stock físico disponible en el catálogo ante el alta de nuevos préstamos.
- Casos que maneja:
 - AFTER INSERT ON PRESTAMO: Cuando un ejemplar es asignado de manera exitosa, el trigger se dispara de forma automática. Identifica el código isbn del libro correspondiente uniendo las tablas dinámicamente, y resta de inmediato 1 unidad a la columna stock_disponible de la tabla LIBRO para mantener actualizado el inventario en estantería.

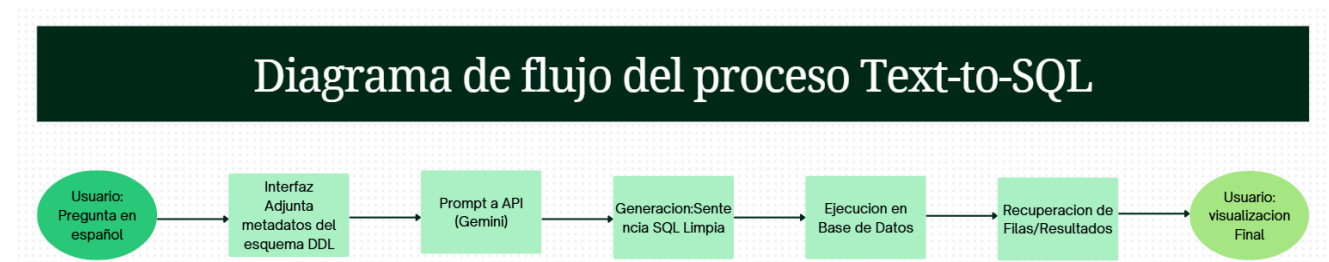
trg_actualizar_stock_update

- Propósito: Sincronizar de manera automatizada el inventario disponible en la tabla LIBRO cuando ocurren modificaciones o actualizaciones en el ciclo de vida de un préstamo activo.
- Casos que maneja:
 - Transición a Devolución : Al ejecutarse un cambio de estado hacia 'DEVUELTO', el disparador identifica dinámicamente mediante una subconsulta el código isbn del EJEMPLAR afectado y ejecuta un comando UPDATE que suma 1 unidad a la columna stock_disponible de la tabla LIBRO, reflejando que la copia física regresa a estar disponible en las estanterías de la biblioteca.
 - Reversión de Devolución : Maneja el caso excepcional o correctivo donde un préstamo previamente marcado como finalizado vuelve a un estado operativo . En este escenario,

el trigger resta de inmediato 1 unidad al atributo stock_disponible del libro correspondiente, previniendo discrepancias de inventario o inconsistencias lógicas frente a auditorías externas.

Arquitectura del agente IA: diagrama de flujo del proceso Text-to-SQL.

Resumen del Flujo Completo



Prompt de sistema utilizado: versión final con explicación de cada sección.

Versión Final del Prompt de Sistema

Sos un traductor de lenguaje natural a SQL para MySQL. Tu única salida debe ser código SQL ejecutable.

Cero explicaciones, cero saludos, cero formato Markdown (no uses ```sql ni ```).

Reglas relacionales estrictas:

1. LIBRO y AUTOR se unen OBLIGATORIAMENTE pasando por la tabla intermedia LIBRO_AUTOR.
2. LIBRO y GENERO se unen OBLIGATORIAMENTE pasando por la tabla intermedia LIBRO_GENERO.

3. En AUTOR y SOCIO, para buscar por nombre y apellido a la vez, usa: CONCAT(nombre, ' ', apellido).

Explicación de cada sección

- **Identidad y Dialecto** : El mandato *"Sos un traductor de lenguaje natural a SQL para MySQL"* restringe el espacio de respuestas al motor relacional utilizado. Esto previene errores de sintaxis al obligar al modelo a usar funciones nativas exclusivas de MySQL.
- **Restricciones de Formato** : La instrucción *"Tu única salida debe ser código SQL ejecutable. Cero explicaciones..."* anula la tendencia conversacional por defecto del LLM.
- **Integridad Referencial** : Las reglas 1 y 2 fuerzan la triangulación a través de las tablas puente (LIBRO_AUTOR y LIBRO_GENERO). Los modelos de lenguaje suelen ignorar estas tablas intermedias al intentar generar JOINS directos entre entidades fuertes, rompiendo la lógica relacional del esquema.
- **Estandarización de Consultas Homónimas**: La regla 3 (CONCAT) unifica el criterio para búsquedas de personas. Cuando el usuario pregunta por un nombre completo, evita que la IA intente procesar filtros condicionales fragmentados o ineficientes.

Evaluación del agente: tabla con las 10+ preguntas, SQL generado, resultado y análisis de errores.

Pregunta 1: "¿Cuáles son los 5 libros más prestados este año?"

```
> # Hacer la pregunta que quieras en español
pregunta = "¿Cuáles son los 5 libros más prestados este año?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📄 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")

(13) ✓ 22.9s Python
```

🤖 Procesando pregunta: '¿Cuáles son los 5 libros más prestados este año?'...

🤖 SQL generado por la IA:

```
SELECT L.titulo, COUNT(P.id_prestamo) AS total_prestamos
FROM PRESTAMO AS P
JOIN EJEMPLAR AS E ON P.id_ejemplar = E.id_ejemplar
JOIN LIBRO AS L ON E.isbn = L.isbn
WHERE YEAR(P.fecha_prestamo) = YEAR(CURDATE())
GROUP BY L.isbn, L.titulo
```

```
ORDER BY total_prestamos DESC
LIMIT 5
```

 Resultados obtenidos:

	titulo	total_prestamos
0	Fundación	3
1	Yo, Robot	3
2	El Resplandor	3
3	It (Eso)	3
4	El Hobbit	3

Pregunta 2: "¿Qué socios tienen préstamos vencidos en este momento?"

```
# Hacé la pregunta que quieras en español
pregunta = "¿Qué socios tienen préstamos vencidos en este momento?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📊 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")
```

😞 Procesando pregunta: '¿Qué socios tienen préstamos vencidos en este momento?'...

 SQL generado por la IA:

```
SELECT DISTINCT nombre_socio, email_socio
FROM vw_prestamos_activos
WHERE dias_de_mora > 0;
```

 Resultados obtenidos:

	nombre_socio	email_socio
0	Juan Pérez	juan@mail.com
1	María Gómez	maria@mail.com
2	Carlos López	carlos@mail.com
3	Ana Martínez	ana@mail.com
4	Pedro Rodríguez	pedro@mail.com
5	Lucía Fernández	lucia@mail.com
6	Jorge García	jorge@mail.com
7	Sofía Díaz	sofia@mail.com
8	Luis Romero	luis@mail.com
9	David Suárez	david@mail.com
10	Paula Muñoz	paula@mail.com
11	Javier Rubio	javier@mail.com
12	Nuria Sanz	nuria@mail.com
13	Mario Iglesias	mario@mail.com
14	Clara Cruz	clara@mail.com
15	Andrés Reyes	andres@mail.com
16	Eva Garrido	eva@mail.com
17	Tomás Herrera	tomas@mail.com
18	Inés Medina	ines@mail.com

Pregunta 3: "¿Cuántos ejemplares disponibles hay del libro con ISBN X?"

```

# Hacé la pregunta que quieras en español
pregunta = "¿Cuántos ejemplares disponibles hay del libro con ISBN X?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📄 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")

```

🤖 Procesando pregunta: '¿Cuántos ejemplares disponibles hay del libro con ISBN X?'...

🤖 SQL generado por la IA:

```
SELECT stock_disponible FROM LIBRO WHERE isbn = 'X'
```

🚫 No se encontraron resultados o la consulta falló.

Pregunta 4: "¿Qué libros de ciencia ficción están disponibles para prestar?"

```

# Hacé la pregunta que quieras en español
pregunta = "¿Qué libros de ciencia ficción están disponibles para prestar?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📄 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")

```

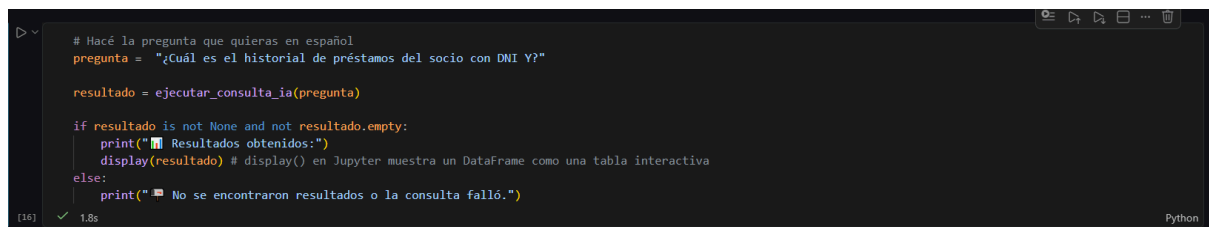
🤔 Procesando pregunta: '¿Qué libros de ciencia ficción están disponibles para prestar?'...

🤖 SQL generado por la IA:

```
SELECT L.titulo
FROM LIBRO L
JOIN LIBRO_GENERO LG ON L.isbn = LG.isbn
JOIN GENERO G ON LG.id_genero = G.id_genero
WHERE G.nombre = 'Ciencia Ficción' AND L.stock_disponible > 0;
```

🚩 No se encontraron resultados o la consulta falló.

Pregunta 5: "¿Cuál es el historial de préstamos del socio con DNI Y?"



```
# Hací la pregunta que quieras en español
pregunta = "¿Cuál es el historial de préstamos del socio con DNI Y?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📄 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚩 No se encontraron resultados o la consulta falló.")
```

🤔 Procesando pregunta: '¿Cuál es el historial de préstamos del socio con DNI Y?'...

🤖 SQL generado por la IA:

```
SELECT
    P.id_prestamo,
    L.titulo AS titulo_libro,
    P.fecha_prestamo,
    P.fecha_vencimiento,
    P.fecha_devolucion,
    P.estado
FROM
    PRESTAMO AS P
JOIN
    SOCIO AS S ON P.id_socio = S.id_socio
JOIN
    EJEMPLAR AS E ON P.id_ejemplar = E.id_ejemplar
JOIN
    LIBRO AS L ON E.isbn = L.isbn
WHERE
    S.dni = 'Y';
```

🚩 No se encontraron resultados o la consulta falló.

Pregunta 6: "¿Qué autores tienen más de 3 libros en la biblioteca?"

```
# Hacé la pregunta que quieras en español
pregunta = "¿Qué autores tienen más de 3 libros en la biblioteca?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📊 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")
```

😞 Procesando pregunta: '¿Qué autores tienen más de 3 libros en la biblioteca?'...

🤖 SQL generado por la IA:
SELECT A.nombre, A.apellido
FROM AUTOR AS A
JOIN LIBRO_AUTOR AS LA ON A.id_autor = LA.id_autor
GROUP BY A.id_autor, A.nombre, A.apellido
HAVING COUNT(LA.isbn) > 3

🚫 No se encontraron resultados o la consulta falló.

pregunta 7: "¿Cuántas sanciones se generaron en el último mes?"

```
# Hacé la pregunta que quieras en español
pregunta = "¿Cuántas sanciones se generaron en el último mes?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📊 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")
```

😞 Procesando pregunta: '¿Cuántas sanciones se generaron en el último mes?'...

🤖 SQL generado por la IA:
SELECT COUNT(*) FROM SANCION WHERE fecha_inicio >= DATE_SUB(CURDATE(),
INTERVAL 1 MONTH)

📊 Resultados obtenidos:

COUNT(*)	
0	2

Pregunta 8: "¿cuantos socios poseen el mismo apellido?"

```
# Hacé la pregunta que quieras en español
pregunta = "¿cuantos socios poseen el mismo apellido?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📄 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")
```

🤖 Procesando pregunta: '¿cuantos socios poseen el mismo apellido?'...

🤖 SQL generado por la IA:
SELECT apellido, COUNT(id_socio) AS cantidad_socios
FROM SOCIO
GROUP BY apellido
HAVING COUNT(id_socio) > 1;

🇲🇪 No se encontraron resultados o la consulta falló.

Pregunta 9: "¿Cual es la nacionalidad con mas libros publicados?"

```
# Hacé la pregunta que quieras en español
pregunta = "¿Cual es la nacionalidad con mas libros publicados?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📄 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")
```

🤖 Procesando pregunta: '¿Cual es la nacionalidad con mas libros publicados?'...

🤖 SQL generado por la IA:
SELECT A.nacionalidad
FROM LIBRO L
JOIN LIBRO_AUTOR LA ON L.isbn = LA.isbn
JOIN AUTOR A ON LA.id_autor = A.id_autor
GROUP BY A.nacionalidad
ORDER BY COUNT(DISTINCT L.isbn) DESC
LIMIT 1

📊 Resultados obtenidos:

nacionalidad
0 Estadounidense

Pregunta 10: "¿Cual es el libro mas popular (con mas prestamos)?"

```
# Hací la pregunta que quieras en español
pregunta = "¿Cual es el libro mas popular (con mas prestamos)?"

resultado = ejecutar_consulta_ia(pregunta)

if resultado is not None and not resultado.empty:
    print("📄 Resultados obtenidos:")
    display(resultado) # display() en Jupyter muestra un DataFrame como una tabla interactiva
else:
    print("🚫 No se encontraron resultados o la consulta falló.")
```

😓 Procesando pregunta: '¿Cual es el libro mas popular (con mas prestamos)?'...

🤖 SQL generado por la IA:

```
SELECT L.titulo
FROM PRESTAMO P
JOIN EJEMPLAR E ON P.id_ejemplar = E.id_ejemplar
JOIN LIBRO L ON E.isbn = L.isbn
GROUP BY L.isbn, L.titulo
ORDER BY COUNT(P.id_prestamo) DESC
LIMIT 1
```

📊 Resultados obtenidos:

titulo
0 Fundación

Módulo de recomendaciones: descripción del algoritmo y ejemplos de salida.

Descripción detallada del algoritmo de recomendación por lecturas anteriores

El módulo de recomendaciones de BibliolA implementa un enfoque híbrido que combina el filtrado basado en el historial transaccional del usuario (obtenido directamente desde el motor relacional MySQL) con las capacidades de razonamiento semántico y generación creativa de un Modelo de Lenguaje Grande (LLM) a través de la API de Google Gemini.

El proceso algorítmico se divide en cuatro fases:

1. Fase de Extracción Transaccional (MySQL): El sistema ejecuta consultas SQL dinámicas y parametrizadas utilizando un cursor de tipo diccionario (dictionary=True).
 - Primero, valida la existencia y estado del socio utilizando su DNI o ID.
 - Segundo, realiza múltiples combinaciones (INNER JOIN) entre las tablas PRESTAMO, EJEMPLAR, LIBRO, GENERO y AUTOR. Mediante la función agregada GROUP_CONCAT(DISTINCT ...), consolida un perfil unificado con los autores y géneros literarios que el socio consumió históricamente.
2. Fase de Filtrado de Candidatos Operativos: El algoritmo aplica reglas de negocio estrictas en una segunda consulta relacional para determinar los libros aptos para la recomendación. Utiliza una cláusula WHERE stock_disponible > 0 para garantizar la disponibilidad física del libro en la estantería y una subconsulta con NOT IN que excluye de forma permanente los títulos que el socio ya leyó, impidiendo recomendaciones redundantes.
3. Fase de Enriquecimiento y Prompt Engineering: Las estructuras de datos (listas de diccionarios de Python) se transforman en cadenas de texto formateadas. El algoritmo construye un prompt con un rol especializado (*Bibliotecario Experto*) e inyecta de manera segura el perfil de gustos del socio junto con el catálogo de candidatos. Para controlar la salida del LLM, se le imponen restricciones de formato rígidas de tres etiquetas fijas.
4. Fase de Inferencia Creativa y Configuración: El prompt estructurado se envía al modelo gemini-2.5-flash. A diferencia de la precisión matemática requerida en el módulo Text-to-SQL, aquí el algoritmo configura el hiperparámetro temperature = 0.7. Este valor le otorga al modelo la flexibilidad lingüística necesaria para redactar justificaciones persuasivas y descripciones visuales detalladas para las portadas de los libros sugeridos.

Descripción detallada del algoritmo de recomendación por socios afines

El módulo de recomendaciones de BibliolA implementa un enfoque de Filtrado Colaborativo basado en Socios Afines, potenciado por las capacidades de análisis semántico de la API de Google Gemini (modelo gemini-2.5-flash). En lugar de analizar al usuario de forma aislada, el algoritmo identifica patrones de consumo cruzados entre diferentes lectores para descubrir joyas literarias ocultas.

El flujo operativo del algoritmo se divide en las siguientes etapas:

1. **Identificación de Socios Afines (Vecindad en MySQL):** El algoritmo ejecuta una consulta relacional avanzada para encontrar otros usuarios que compartan los mismos gustos que el socio actual. Para ello, toma los isbn de los libros que el socio ha pedido prestados y busca en la tabla PRESTAMO qué otros id_socio solicitaron esos mismos títulos. Los socios que tengan la mayor cantidad de coincidencias son clasificados como "socios afines".
2. **Extracción y Filtrado del Catálogo Candidato:** Una vez identificada la comunidad de socios afines, el sistema extrae cuáles fueron los *otros* libros que ellos leyeron pero que el socio actual todavía no ha tomado prestados. Esta lista de candidatos se filtra bajo dos restricciones estrictas del DDL:
 - Se verifica que los ejemplares tengan stock_disponible > 0 en la tabla LIBRO.
 - Se excluyen los libros con estado_fisico = 'BAJA' en la tabla EJEMPLAR.
3. **Construcción del Contexto para la IA:** El script toma la lista de libros recomendados por los socios afines (con sus respectivos títulos, autores y géneros) y construye el prompt. Le asigna a Gemini el rol de *Bibliotecario Experto*, pero con una nueva instrucción de ingeniería de prompts: debe justificar la recomendación explicando la afinidad del círculo de lectores (ej. "*Otros usuarios con tus mismos gustos leyeron...*").
4. **Inferencia y Configuración Creativa:** El prompt se envía a la API con una temperature = 0.7. Esto le permite al modelo gemini-2.5-flash redactar reseñas atractivas y diseñar conceptualmente portadas llamativas para los libros sugeridos, manteniendo la estructura rígida de salida en tres etiquetas (TÍTULO, PORTADA y JUSTIFICACIÓN).

ejemplo:

```
obtener_recomendaciones_socio(11111111, tipo_busqueda='dni')  
recomendacion_lectores_afines(11111111, tipo_busqueda='dni')
```

Buscando historial y armando recomendaciones para el socio (dni): 11111111...

RECOMENDACIONES PARA JUAN PÉREZ

=====

¡Hola, Juan Pérez! Es un placer verte de nuevo por aquí. He estado revisando tu historial de lectura y veo que tienes un gusto muy particular por las narrativas que invitan a la reflexión y los mundos bien contruidos, ya sea en la ciencia ficción o en la fantasía.

Basándome en tu aprecio por la complejidad de "Fundación" de Asimov y la originalidad de "Ficciones" de Borges, he seleccionado tres títulos de nuestra colección que creo que podrían interesarte mucho:

TÍTULO: Rayuela

PORTADA: Una silueta abstracta de una figura humana saltando sobre un tablero de ajedrez o un laberinto de calles parisinas, con tonos sepia y un toque de azul melancólico. Las letras del título, en una tipografía elegante pero ligeramente desordenada, parecen flotar o caer, sugiriendo la naturaleza fragmentada y lúdica de la obra.

JUSTIFICACIÓN: Juan, viendo que disfrutó de la profundidad y la experimentación literaria de "Ficciones" de Jorge Luis Borges, creo que "Rayuela" de Julio Cortázar le resultará fascinante. Ambos autores comparten una maestría en la prosa y una inclinación por desafiar las convenciones narrativas. Si le gustó la forma en que Borges juega con la realidad y la ficción, la estructura no lineal y las múltiples lecturas de "Rayuela" le ofrecerán una experiencia igualmente enriquecedora y estimulante, sumergiéndose en un tipo de fantasía más existencial y onírica.

TÍTULO: El Imperio Final

PORTADA: Un paisaje desolado y ceniciento bajo un cielo rojizo, con una figura solitaria y encapuchada en primer plano, quizás con un puñado de monedas metálicas levitando a su alrededor. Al fondo, una imponente fortaleza oscura se alza, emanando una sensación de opresión y misterio. El título, en letras robustas y ligeramente desgastadas, se asienta en la parte inferior.

JUSTIFICACIÓN: Sabiendo que se sumergió en el vasto y complejo universo de "Fundación" de Isaac Asimov, le sugiero "El Imperio Final" de Brandon Sanderson. Aunque cambia la ciencia ficción por la fantasía épica, Sanderson es un maestro en la construcción de mundos detallados y sistemas de magia intrincados, algo que sin duda apreciará si le gustó la coherencia y la escala de la visión de Asimov. La intriga política, los personajes bien desarrollados y un misterio central que se desvela poco a poco le recordarán la profundidad estratégica de la saga de la Fundación.

TÍTULO: Juego de Tronos

PORTADA: Un trono de hierro forjado con espadas, imponente y oscuro, bajo un cielo tormentoso. Un lobo huargo o un dragón estilizado podría aparecer sutilmente

en el fondo, sugiriendo las casas nobles y la magia latente. Los colores son sombríos, predominando los grises, negros y rojos sangre, evocando la brutalidad y la seriedad de la historia.

JUSTIFICACIÓN: Dado su interés en la fantasía con "Ficciones" y su aprecio por narrativas complejas como "Fundación", "Juego de Tronos" de George R.R. Martin es una elección excelente. Esta obra le sumergirá en un mundo de fantasía épica con una profundidad política y una intriga que rivalizan con la estrategia de Asimov, pero ambientada en un reino medieval fantástico. Si le atraen los personajes moralmente ambiguos, las tramas intrincadas y un mundo ricamente detallado, esta saga le ofrecerá horas de lectura apasionantes.

Conclusiones: lecciones aprendidas, limitaciones del agente, mejoras posibles.

- **Conclusión general:** dado que este trabajo requería de la implementación de varias tecnologías con las cuales nunca habíamos trabajado antes, como la implementación de librerías de python y el uso del entorno de jupyter, tuvimos que llevar a cabo un proceso de aprendizaje y de autoenseñanza para así poder hacer uso de estas herramientas.

La lección aprendida más importante(y uno de nuestros mayores problemas) fue acerca de la distribución del trabajo y sobre la organización del tiempo de cada uno de los integrantes, dado que este trabajo se realizó en épocas de parciales, tuvimos que aprender a organizarnos y comunicarnos para así distribuirnos el trabajo de realizar este proyecto.

Con respecto al modelo de inteligencia artificial, no tuvimos en ningún momento grandes inconvenientes con el agente en sí, las únicas limitaciones que tuvimos que enfrentar fueron acerca de la dificultad de su implementación y sobre la escasa cantidad de tokens que podíamos gastar para hacer consultas. Con respecto a todo lo demás, el agente de gemini fue una herramienta más que completa.

Una posible mejora que pensamos como grupo, fue sobre la expansión de la base de datos, más específicamente sobre la agregación de más atributos en las tablas de socio,libro,ejemplar para así poder mejorar el sistema de recomendación de libros y de lectores afines. Creemos que con estas implementaciones la IA podría hacer recomendaciones más precisas y más útiles (como por ejemplo, recomendar lectores afines con una edad similar).

Bibliografía y recursos utilizados.

- Gemini
- Jupyter: <https://docs.jupyter.org/en/latest/>
- Apuntes de la cátedra

- Python
- VScode